



```
In [7]: import librosa
import librosa.display
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import random as rd
from pathlib import Path

AUDIO_DIR = Path("Ressources")

n_fft = 1024
hop_length = 64
filename = "Ressources/ValabFr-20221128_133055.735.flac"

detections_csv = "Ressources/detections.csv"

df = pd.read_csv(detections_csv)

target = str(Path(filename))
df["filename"] = df["filename"].apply(lambda f: str(Path("Ressources") / Path(

result = df[df["filename"] == target]

if result.empty:
    raise ValueError(f"Aucune détection pour {target}")

row = result.iloc[0]

#Question 1

def load_signal(filename):
    signal, frequence_echantillonnage = librosa.load(filename, sr=None)
    # frequence_echantillonnage = librosa.get_samplerate(filename)
    # print("Fréquence d'échantillonnage : ", frequence_echantillonnage, "Hertz")
    return signal, frequence_echantillonnage

def get_spectro(signal, type):
    return np.abs(librosa.stft(signal, window=type))

def show_spectro(sr, spectre):
    spectre_db = librosa.amplitude_to_db(spectre)

    plt.figure(figsize=(12, 6))

    librosa.display.specshow(
        spectre_db,
        sr=sr,
        y_axis="log",
        x_axis="time",
    )
```

```

plt.title("Spectrogramme STFT")
plt.colorbar(format="%+2.0f dB")
plt.tight_layout()
plt.show()

signal, frequency_echantillonage = load_signal(filename)
spectre = get_spectro(signal, "hann")

def load_signal(filename, offset=0, duration=10):
    signal, frequency_echantillonage = librosa.load(filename, offset=offset, duration=duration)
    return signal, frequency_echantillonage

def get_spectro(signal, frequency_echantillonage, fmin, fmax, n_fft, hop_length):
    signal = np.abs(librosa.stft(signal, n_fft=n_fft, hop_length=hop_length, window='hann'))
    tab_freq = librosa.fft_frequencies(sr=frequency_echantillonage, n_fft=n_fft)

    indice_min = np.searchsorted(tab_freq, fmin, 'left')
    indice_max = np.searchsorted(tab_freq, fmax, 'right')

    spectre = signal[indice_min:indice_max, :]

    return spectre

def show_spectro(sr, spectre):
    spectre_db = librosa.amplitude_to_db(spectre)

    plt.figure(figsize=(12, 6))

    librosa.display.specshow(
        spectre_db,
        sr=sr,
        x_axis="time",
        y_axis="hz"
    )

    plt.title("Spectrogramme STFT")
    plt.tight_layout()
    plt.show()

nv_signal, frequency_echantillonage = load_signal(filename)
nv_spectre = get_spectro(nv_signal, frequency_echantillonage, 100, 150, n_fft, hop_length)

offset = row.iloc[2]
start = row.iloc[3]
fmin = row.iloc[4] - 5
fmax = row.iloc[5] + 5
duration = start - offset

nv_signal, frequency_echantillonage = load_signal(
    filename=target,

```

```

        offset=offset,
        duration=duration
    )

nv_spectre = get_spectro(
    nv_signal,
    frequence_echantillonage,
    fmin,
    fmax,
    1024,
    hop_length,
    "hann"
)

N_FFT = 1024
HOP_LENGTH = 64
WINDOW_DURATION = 2.0
FMIN = 120
FMAX = 130
WINDOW = "hann"

def shift_signal(signal, pulse_offset, pulse_duration, window_duration, fs):
    pulse_samples = int(pulse_duration * fs)
    window_samples = int(window_duration * fs)
    signal_samples = len(signal)

    max_start_in_window = window_samples - pulse_samples
    pulse_pos_in_window = rd.randint(0, max_start_in_window)

    window_start = int(pulse_offset * fs) - pulse_pos_in_window
    window_start = max(0, min(signal_samples - window_samples, window_start))

    return signal[window_start : window_start + window_samples]

def get_pulses(target, n_fft, hop_length, window_duration):
    df = pd.read_csv(detections_csv)
    result = df[df["filename"] == target]

    all_pulses = []

    signal_complet, freq_complet = load_signal(filename=target, offset=0, dura

    for _, row in result.iterrows():
        offset = row["Begin Time (s)"]
        start = row["End Time (s)"]
        fmin = row["Low Freq (Hz)"] - 5
        fmax = row["High Freq (Hz)"] + 5
        duration = start - offset

        nv_signal = shift_signal(
            signal=signal_complet,
            pulse_offset=offset,

```

```

        pulse_duration=duration,
        window_duration=window_duration,
        fs=freq_complet
    )

    nv_spectre = get_spectro(
        nv_signal,
        freq_complet,
        fmin,
        fmax,
        n_fft,
        hop_length,
        "hann"
    )

    all_pulses.append(nv_spectre)
return all_pulses

def get_spectros_pos_one_file(filename, df, window_duration=WINDOW_DURATION, n
result = df[df["filename"] == filename]
if result.empty:
    return np.empty((0, 0, 0))

signal, sr = load_signal(filename, offset=0, duration=None)
spectros = []

for _, row in result.iterrows():
    offset = row["Begin Time (s)"]
    duration = row["End Time (s)"] - offset

    nv_signal = shift_signal(signal, offset, duration, window_duration, sr)
    nv_spectre = get_spectro(nv_signal, sr, FMIN, FMAX, n_fft, hop_length,

    spectros.append(nv_spectre)

return np.stack(spectros, axis=0)

def get_all_spectros_pos(df, window_duration=WINDOW_DURATION, n_fft=N_FFT, hop
all_spectros = []

for filename in df["filename"].unique():
    spectros = get_spectros_pos_one_file(
        filename, df, window_duration, n_fft, hop_length
    )
    if spectros.size > 0:
        all_spectros.append(spectros)

return np.concatenate(all_spectros, axis=0)

def generate_negative(filename, df, window_duration=WINDOW_DURATION, n_fft=N_F
signal, sr = load_signal(filename, offset=0, duration=None)

```

[illegible]

```

all_spectros_neg = []

for filename in df["filename"].unique():
    n_pos = len(df[df["filename"] == filename])
    spectros_neg = get_spectros_neg_one_file(filename, df, n_pos,
                                              window_duration,
                                              fmin, fmax,
                                              n_fft, hop_length)

    if spectros_neg.size > 0:
        all_spectros_neg.append(spectros_neg)

if all_spectros_neg:
    return np.concatenate(all_spectros_neg, axis=0)
else:
    return np.empty((0,0,0))

def generate_dataset(spectros_pos, spectros_neg, output_dir='.'):
    if spectros_pos.size == 0 or spectros_neg.size == 0:
        raise ValueError("Empty positive or negative spectrograms")

    X = np.concatenate([spectros_pos, spectros_neg], axis=0)
    Y = np.concatenate([
        np.ones(len(spectros_pos), dtype=np.int32),
        np.zeros(len(spectros_neg), dtype=np.int32)
    ])

    permutation = np.random.permutation(len(X))
    X = X[permutation]
    Y = Y[permutation]

    output_path = Path(output_dir)
    output_path.mkdir(parents=True, exist_ok=True)

    np.save(output_path / 'x.npy', X)
    np.save(output_path / 'y.npy', Y)

    print()
    print(f"Dataset généré : X={X.shape}, Y={Y.shape}")
    print(f"Positifs: {Y.sum()}, Négatifs: {len(Y) - Y.sum()}")

    return X, Y

# spectros_pos = get_all_spectros_pos(df)
# spectros_neg = get_all_spectros_neg(df)

# X, Y = generate_dataset(spectros_pos, spectros_neg)

def get_spectros_neg_one_file(filename, df, N,
                              window_duration=WINDOW_DURATION,
                              fmin=FMIN, fmax=FMAX,
                              n_fft=N_FFT, hop_length=HOP_LENGTH):
    spectros = []
    for _ in range(N):

```

```

        spectro = generate_negative(filename, df,
                                    window_duration=window_duration,
                                    n_fft=n_fft,
                                    hop_length=hop_length)

    if spectro is not None:
        spectros.append(spectro)

if spectros:
    return np.stack(spectros, axis=0)
else:
    return np.empty((0,0,0))

def get_all_spectros_neg(df, K_shifts,
                        window_duration=WINDOW_DURATION,
                        fmin=FMIN, fmax=FMAX,
                        n_fft=N_FFT, hop_length=HOP_LENGTH):
    all_spectros_neg = []

    for filename in df["filename"].unique():
        n_pos = len(df[df["filename"] == filename])

        N_samples = n_pos * K_shifts

        spectros_neg = get_spectros_neg_one_file(
            filename, df, N_samples,
            window_duration,
            fmin, fmax,
            n_fft, hop_length
        )
        if spectros_neg.size > 0:
            all_spectros_neg.append(spectros_neg)

    if all_spectros_neg:
        return np.concatenate(all_spectros_neg, axis=0)
    else:
        return np.empty((0,0,0))

def get_spectros_pos_one_file(filename, df, K_shifts, window_duration=WINDOW_D
    result = df[df["filename"] == filename]
    if result.empty:
        return np.empty((0, 0, 0))

    signal, sr = load_signal(filename, offset=0, duration=None)
    spectros = []

    for _, row in result.iterrows():
        offset = row["Begin Time (s)"]
        duration = row["End Time (s)"] - offset

        for _ in range(K_shifts):
            nv_signal = shift_signal(signal, offset, duration, window_duration

```

```

        nv_spectre = get_spectro(nv_signal, sr, FMIN, FMAX, n_fft, hop_length)
        spectros.append(nv_spectre)

    return np.stack(spectros, axis=0)

def get_all_spectros_pos(df, K_shifts, window_duration=WINDOW_DURATION, n_fft=
all_spectros = []

    for filename in df["filename"].unique():
        spectros = get_spectros_pos_one_file(
            filename, df, K_shifts, window_duration, n_fft, hop_length
        )
        if spectros.size > 0:
            all_spectros.append(spectros)

    return np.concatenate(all_spectros, axis=0)

K_SHIFTS = 3

spectros_pos = get_all_spectros_pos(df, K_SHIFTS)
spectros_neg = get_all_spectros_neg(df, K_SHIFTS)

X, Y = generate_dataset(spectros_pos, spectros_neg)

```

Dataset généré : X=(636, 3, 101), Y=(636,)
Positifs: 318, Négatifs: 318

```

In [8]: import pandas as pd
        from sklearn.model_selection import train_test_split
        import torch
        import torch.nn as nn
        import numpy as np
        from pathlib import Path
        from torch.utils.data import TensorDataset, DataLoader
        from torch.optim import AdamW
        from torch.optim.lr_scheduler import OneCycleLR
        import math
        import matplotlib.pyplot as plt

        history = {
            'train_loss': [],
            'train_acc': [],
            'train_f1': [],
            'valid_loss': [],
            'valid_acc': [],
            'valid_f1': [],
            'grad_norm': []
        }

        # Question 1 : Standardisation par bande de fréquence
        def calc_mean_std(X_train):
            mean = X_train.mean(axis=(0, 2), keepdims=True)

```

```

std = X_train.std(axis=(0, 2), keepdims=True)
std[std == 0] = 1.0
return mean, std

def standardisation(X, mean, std):
    return (X - mean) / std

# 2. Les pulses acoustiques sont dans une gamme de fréquence spécifique c'est
# Question 2 : Construction des ensembles train/valid/test

def create_train_valid_test_splits(csv_path, output_base_dir='dataset'):
    df = pd.read_csv(csv_path)
    df["filename"] = df["filename"].apply(lambda f: str(Path("Ressources") / f))

    files = df["filename"].unique()
    train_files, temp_files = train_test_split(files, test_size=0.6, random_state=42)
    valid_files, test_files = train_test_split(temp_files, test_size=0.5, random_state=42)

    splits = {'train': train_files, 'valid': valid_files, 'test': test_files}
    output_base = Path(output_base_dir)

    for split_name, split_files in splits.items():
        df_split = df[df["filename"].isin(split_files)]
        spectros_pos = get_all_spectros_pos(df_split, K_SHIFTS)
        spectros_neg = get_all_spectros_neg(df_split, K_SHIFTS)
        X, Y = generate_dataset(spectros_pos, spectros_neg, output_dir=str(output_base / split_name))
        print(f"{split_name}: {len(X)} samples ({Y.sum()} positifs, {len(Y) - Y.sum()} négatifs)")

    X_train = np.load(output_base / 'train' / 'x.npy')
    X_valid = np.load(output_base / 'valid' / 'x.npy')
    X_test = np.load(output_base / 'test' / 'x.npy')

    mean, std = calc_mean_std(X_train)

    X_train = standardisation(X_train, mean, std)
    X_valid = standardisation(X_valid, mean, std)
    X_test = standardisation(X_test, mean, std)

    np.save(output_base / 'train' / 'x.npy', X_train)
    np.save(output_base / 'valid' / 'x.npy', X_valid)
    np.save(output_base / 'test' / 'x.npy', X_test)

create_train_valid_test_splits(detections_csv, output_base_dir='dataset')

# Question 3 : MLP

class MyMLP(nn.Module):
    def __init__(self, input_size, hidden_sizes=[128, 64, 32, 16], dropout_rate=0.5):
        super().__init__()

```

```

self.layers = nn.ModuleList()
self.activations = nn.ModuleList()
self.dropouts = nn.ModuleList()

layer_sizes = [input_size] + hidden_sizes

for i in range(len(layer_sizes) - 1):
    n_in = layer_sizes[i]
    n_out = layer_sizes[i + 1]

    linear = nn.Linear(n_in, n_out)

    a = math.sqrt(6.0 / (n_in + n_out))
    nn.init.uniform_(linear.weight, -a, a)
    nn.init.zeros_(linear.bias)

    self.layers.append(linear)
    self.activations.append(nn.ReLU())
    self.dropouts.append(nn.Dropout(dropout_rate if i < 2 else 0.0))

self.output_layer = nn.Linear(layer_sizes[-1], 1)
a_out = math.sqrt(6.0 / (layer_sizes[-1] + 1))
nn.init.uniform_(self.output_layer.weight, -a_out, a_out)
nn.init.zeros_(self.output_layer.bias)

def forward(self, x):
    batch_size = x.size(0)
    x = x.view(batch_size, -1)

    for layer, activation, dropout in zip(self.layers, self.activations, s
        x = layer(x)
        x = activation(x)
        x = dropout(x)

    x = self.output_layer(x)
    return x.squeeze(-1)

```

Question 4 : Chargement dataset avec PyTorch

```

def load_dataset(data_dir):
    data_path = Path(data_dir)
    X = np.load(data_path / 'x.npy')
    Y = np.load(data_path / 'y.npy')
    X_tensor = torch.from_numpy(X).float()
    Y_tensor = torch.from_numpy(Y).float()
    return TensorDataset(X_tensor, Y_tensor)

```

Question 6 : Métriques d'évaluation

```

def compute_metrics(y_true, y_pred):
    y_pred = y_pred.int()

```

```

y_true = y_true.int()

TP = ((y_pred == 1) & (y_true == 1)).sum().item()
TN = ((y_pred == 0) & (y_true == 0)).sum().item()
FP = ((y_pred == 1) & (y_true == 0)).sum().item()
FN = ((y_pred == 0) & (y_true == 1)).sum().item()

total = TP + TN + FP + FN
accuracy = (TP + TN) / total if total > 0 else 0.0
precision = TP / (TP + FP) if (TP + FP) > 0 else 0.0
recall = TP / (TP + FN) if (TP + FN) > 0 else 0.0
f1 = 2 * precision * recall / (precision + recall) if (precision + recall)
fpr = FP / (FP + TN) if (FP + TN) > 0 else 0.0
fnr = FN / (FN + TP) if (FN + TP) > 0 else 0.0

return {
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1': f1,
    'fpr': fpr,
    'fnr': fnr
}

```

```

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss = 0.0
    all_preds = []
    all_labels = []

    for X_batch, Y_batch in loader:
        X_batch = X_batch.to(device)
        Y_batch = Y_batch.to(device)

        outputs = model(X_batch)
        loss = criterion(outputs, Y_batch)
        total_loss += loss.item()

        preds = (torch.sigmoid(outputs) > 0.5)
        all_preds.append(preds.cpu())
        all_labels.append(Y_batch.cpu())

    all_preds = torch.cat(all_preds)
    all_labels = torch.cat(all_labels)

    metrics = compute_metrics(all_labels, all_preds)
    metrics['loss'] = total_loss / len(loader)
    return metrics

```

Question 5 : Boucle d'entraînement

```

def train_epoch(model, loader, criterion, optimizer, scheduler, device):
    model.train()
    total_loss = 0.0
    total_grad_norm = 0.0
    n_batches = 0

    for X_batch, Y_batch in loader:
        X_batch = X_batch.to(device)
        Y_batch = Y_batch.to(device)

        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, Y_batch)
        loss.backward()

        grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        total_grad_norm += grad_norm.item()
        n_batches += 1

        optimizer.step()
        scheduler.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(loader)
    avg_grad_norm = total_grad_norm / n_batches

    return avg_loss, avg_grad_norm

# Hyperparamètres
EPOCHS = 100
BATCH_SIZE = 16
LEARNING_RATE = 3e-5

train_dataset = load_dataset('dataset/train')
valid_dataset = load_dataset('dataset/valid')
test_dataset = load_dataset('dataset/test')

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
valid_loader = DataLoader(valid_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

sample_shape = train_dataset[0][0].shape
input_size = int(np.prod(sample_shape))
model = MyMLP(input_size=input_size, hidden_sizes=[128, 64, 32, 16], dropout_r=0.5)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)

criterion = nn.BCEWithLogitsLoss(pos_weight=torch.tensor([1.0], device=device))
optimizer = AdamW(model.parameters(), lr=LEARNING_RATE, weight_decay=1e-3)
scheduler = OneCycleLR(

```

```

optimizer,
max_lr=LEARNING_RATE * 3,
epochs=EPOCHS,
steps_per_epoch=len(train_loader),
pct_start=0.2,
anneal_strategy='cos'
)

print()
print(f"Device: {device}")
print(f"Input size: {input_size}")
print(f"Train: {len(train_dataset)} | Valid: {len(valid_dataset)} | Test: {len(test_dataset)}")
print(f"Architecture: 4 couches linéaires {[128, 64, 32, 16]}")
print(f"Dropout: 0.4 | LR: {LEARNING_RATE} | Batch: {BATCH_SIZE}\n")

best_valid_f1 = 0.0
patience = 15
patience_counter = 0
best_model_path = 'best_model.pth'

for epoch in range(EPOCHS):
    train_loss, grad_norm = train_epoch(model, train_loader, criterion, optimizer, device)
    train_metrics = evaluate(model, train_loader, criterion, device)
    valid_metrics = evaluate(model, valid_loader, criterion, device)

    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_metrics['accuracy'])
    history['train_f1'].append(train_metrics['f1'])
    history['valid_loss'].append(valid_metrics['loss'])
    history['valid_acc'].append(valid_metrics['accuracy'])
    history['valid_f1'].append(valid_metrics['f1'])
    history['grad_norm'].append(grad_norm)

    if valid_metrics['f1'] > best_valid_f1:
        best_valid_f1 = valid_metrics['f1']
        patience_counter = 0
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'valid_f1': best_valid_f1,
        }, best_model_path)
        print(f" → Meilleur modèle sauvegardé (F1={best_valid_f1:.4f})")
    else:
        patience_counter += 1

    if (epoch + 1) % 5 == 0 or epoch < 10:
        print(f"Epoch {epoch+1}/{EPOCHS}")
        print(f"Train - Loss: {train_loss:.4f} | Acc: {train_metrics['accuracy']:.3f} | Prec: {train_metrics['precision']:.3f} | Rec: {train_metrics['recall']:.3f} | F1: {train_metrics['f1']:.3f} | FPR: {train_metrics['fpr']:.3f}")
        print(f"Valid - Loss: {valid_metrics['loss']:.4f} | Acc: {valid_metrics['accuracy']:.3f} | Prec: {valid_metrics['precision']:.3f} | Rec: {valid_metrics['recall']:.3f} | F1: {valid_metrics['f1']:.3f} | FPR: {valid_metrics['fpr']:.3f}")

```

```

        f"F1: {valid_metrics['f1']:.3f} | FPR: {valid_metrics['fpr']:.3f}
print(f"    Best Valid F1: {best_valid_f1:.3f} | Patience: {patience_cou

    if patience_counter >= patience:
        print(f"Early stopping à l'epoch {epoch+1}\n")
        break

print("="*70)
print("ÉVALUATION FINALE SUR TEST SET")
print("="*70)
test_metrics = evaluate(model, test_loader, criterion, device)

print(f"Loss: {test_metrics['loss']:.4f}")
print(f"Accuracy: {test_metrics['accuracy']:.3f}")
print(f"Precision: {test_metrics['precision']:.3f}")
print(f"Recall: {test_metrics['recall']:.3f}")
print(f"F1-Score: {test_metrics['f1']:.3f}")
print(f"FPR: {test_metrics['fpr']:.3f}")
print(f"FNR: {test_metrics['fnr']:.3f}")

def plot_training_history(history):
    epochs = range(1, len(history['train_loss']) + 1)

    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    # Loss
    axes[0].plot(epochs, history['train_loss'], 'b-', label='Train', linewidth=2)
    axes[0].plot(epochs, history['valid_loss'], 'r-', label='Valid', linewidth=2)
    axes[0].set_xlabel('Epoch', fontsize=12)
    axes[0].set_ylabel('Loss', fontsize=12)
    axes[0].set_title('Loss pendant l\'entraînement', fontsize=14, fontweight='bold')
    axes[0].legend(fontsize=11)
    axes[0].grid(True, alpha=0.3)

    # Accuracy
    axes[1].plot(epochs, history['train_acc'], 'b-', label='Train', linewidth=2)
    axes[1].plot(epochs, history['valid_acc'], 'r-', label='Valid', linewidth=2)
    axes[1].set_xlabel('Epoch', fontsize=12)
    axes[1].set_ylabel('Accuracy', fontsize=12)
    axes[1].set_title('Accuracy pendant l\'entraînement', fontsize=14, fontweight='bold')
    axes[1].legend(fontsize=11)
    axes[1].grid(True, alpha=0.3)
    axes[1].set_ylim([0, 1])

    # F1-Score
    axes[2].plot(epochs, history['train_f1'], 'b-', label='Train', linewidth=2)
    axes[2].plot(epochs, history['valid_f1'], 'r-', label='Valid', linewidth=2)
    axes[2].set_xlabel('Epoch', fontsize=12)
    axes[2].set_ylabel('F1-Score', fontsize=12)
    axes[2].set_title('F1-Score pendant l\'entraînement', fontsize=14, fontweight='bold')
    axes[2].legend(fontsize=11)
    axes[2].grid(True, alpha=0.3)

```

```
axes[2].set_ylim([0, 1])

plt.tight_layout()
plt.show()

# Afficher les graphiques
plot_training_history(history)
```

Dataset généré : X=(294, 3, 101), Y=(294,)
Positifs: 147, Négatifs: 147
train: 294 samples (147 positifs, 147 négatifs)

Dataset généré : X=(132, 3, 101), Y=(132,)
Positifs: 66, Négatifs: 66
valid: 132 samples (66 positifs, 66 négatifs)

Dataset généré : X=(210, 3, 101), Y=(210,)
Positifs: 105, Négatifs: 105
test: 210 samples (105 positifs, 105 négatifs)

Device: cpu
Input size: 303
Train: 294 | Valid: 132 | Test: 210
Architecture: 4 couches linéaires [128, 64, 32, 16]
Dropout: 0.4 | LR: 3e-05 | Batch: 16

→ Meilleur modèle sauvegardé (F1=0.1026)

Epoch 1/100

Train - Loss: 0.8882 | Acc: 0.449 | Prec: 0.241 | Rec: 0.048 | F1: 0.080 | FP
R: 0.150 | FNR: 0.952
Valid - Loss: 0.7910 | Acc: 0.470 | Prec: 0.333 | Rec: 0.061 | F1: 0.103 | FP
R: 0.121 | FNR: 0.939
Best Valid F1: 0.103 | Patience: 0/15

Epoch 2/100

Train - Loss: 0.9243 | Acc: 0.449 | Prec: 0.241 | Rec: 0.048 | F1: 0.080 | FP
R: 0.150 | FNR: 0.952
Valid - Loss: 0.7878 | Acc: 0.470 | Prec: 0.333 | Rec: 0.061 | F1: 0.103 | FP
R: 0.121 | FNR: 0.939
Best Valid F1: 0.103 | Patience: 1/15

Epoch 3/100

Train - Loss: 0.8439 | Acc: 0.452 | Prec: 0.267 | Rec: 0.054 | F1: 0.090 | FP
R: 0.150 | FNR: 0.946
Valid - Loss: 0.7834 | Acc: 0.470 | Prec: 0.333 | Rec: 0.061 | F1: 0.103 | FP
R: 0.121 | FNR: 0.939
Best Valid F1: 0.103 | Patience: 2/15

→ Meilleur modèle sauvegardé (F1=0.1250)

Epoch 4/100

Train - Loss: 0.8907 | Acc: 0.452 | Prec: 0.267 | Rec: 0.054 | F1: 0.090 | FP
R: 0.150 | FNR: 0.946
Valid - Loss: 0.7772 | Acc: 0.470 | Prec: 0.357 | Rec: 0.076 | F1: 0.125 | FP
R: 0.136 | FNR: 0.924
Best Valid F1: 0.125 | Patience: 0/15

→ Meilleur modèle sauvegardé (F1=0.1481)

Epoch 5/100

Train - Loss: 0.8491 | Acc: 0.476 | Prec: 0.394 | Rec: 0.088 | F1: 0.144 | FP
R: 0.136 | FNR: 0.912
Valid - Loss: 0.7688 | Acc: 0.477 | Prec: 0.400 | Rec: 0.091 | F1: 0.148 | FP
R: 0.136 | FNR: 0.909

Best Valid F1: 0.148 | Patience: 0/15

→ Meilleur modèle sauvegardé (F1=0.1750)

Epoch 6/100

Train - Loss: 0.8445 | Acc: 0.507 | Prec: 0.526 | Rec: 0.136 | F1: 0.216 | FP
R: 0.122 | FNR: 0.864

Valid - Loss: 0.7575 | Acc: 0.500 | Prec: 0.500 | Rec: 0.106 | F1: 0.175 | FP
R: 0.106 | FNR: 0.894

Best Valid F1: 0.175 | Patience: 0/15

→ Meilleur modèle sauvegardé (F1=0.2000)

Epoch 7/100

Train - Loss: 0.8371 | Acc: 0.520 | Prec: 0.571 | Rec: 0.163 | F1: 0.254 | FP
R: 0.122 | FNR: 0.837

Valid - Loss: 0.7435 | Acc: 0.515 | Prec: 0.571 | Rec: 0.121 | F1: 0.200 | FP
R: 0.091 | FNR: 0.879

Best Valid F1: 0.200 | Patience: 0/15

→ Meilleur modèle sauvegardé (F1=0.2619)

Epoch 8/100

Train - Loss: 0.7965 | Acc: 0.588 | Prec: 0.760 | Rec: 0.259 | F1: 0.386 | FP
R: 0.082 | FNR: 0.741

Valid - Loss: 0.7263 | Acc: 0.530 | Prec: 0.611 | Rec: 0.167 | F1: 0.262 | FP
R: 0.106 | FNR: 0.833

Best Valid F1: 0.262 | Patience: 0/15

→ Meilleur modèle sauvegardé (F1=0.3256)

Epoch 9/100

Train - Loss: 0.7610 | Acc: 0.633 | Prec: 0.791 | Rec: 0.361 | F1: 0.495 | FP
R: 0.095 | FNR: 0.639

Valid - Loss: 0.7060 | Acc: 0.561 | Prec: 0.700 | Rec: 0.212 | F1: 0.326 | FP
R: 0.091 | FNR: 0.788

Best Valid F1: 0.326 | Patience: 0/15

→ Meilleur modèle sauvegardé (F1=0.5361)

Epoch 10/100

Train - Loss: 0.7127 | Acc: 0.711 | Prec: 0.869 | Rec: 0.497 | F1: 0.632 | FP
R: 0.075 | FNR: 0.503

Valid - Loss: 0.6824 | Acc: 0.659 | Prec: 0.839 | Rec: 0.394 | F1: 0.536 | FP
R: 0.076 | FNR: 0.606

Best Valid F1: 0.536 | Patience: 0/15

→ Meilleur modèle sauvegardé (F1=0.5882)

→ Meilleur modèle sauvegardé (F1=0.7304)

→ Meilleur modèle sauvegardé (F1=0.7731)

→ Meilleur modèle sauvegardé (F1=0.8346)

→ Meilleur modèle sauvegardé (F1=0.8548)

Epoch 15/100

Train - Loss: 0.5271 | Acc: 0.901 | Prec: 0.954 | Rec: 0.844 | F1: 0.895 | FP
R: 0.041 | FNR: 0.156

Valid - Loss: 0.5272 | Acc: 0.864 | Prec: 0.914 | Rec: 0.803 | F1: 0.855 | FP
R: 0.076 | FNR: 0.197

Best Valid F1: 0.855 | Patience: 0/15

→ Meilleur modèle sauvegardé (F1=0.8571)

Epoch 20/100

Train - Loss: 0.3707 | Acc: 0.922 | Prec: 0.992 | Rec: 0.850 | F1: 0.916 | FPR: 0.007 | FNR: 0.150

Valid - Loss: 0.3736 | Acc: 0.864 | Prec: 0.900 | Rec: 0.818 | F1: 0.857 | FPR: 0.091 | FNR: 0.182

Best Valid F1: 0.857 | Patience: 3/15

Epoch 25/100

Train - Loss: 0.2658 | Acc: 0.929 | Prec: 0.992 | Rec: 0.864 | F1: 0.924 | FPR: 0.007 | FNR: 0.136

Valid - Loss: 0.3346 | Acc: 0.856 | Prec: 0.898 | Rec: 0.803 | F1: 0.848 | FPR: 0.091 | FNR: 0.197

Best Valid F1: 0.857 | Patience: 8/15

Epoch 30/100

Train - Loss: 0.2243 | Acc: 0.939 | Prec: 0.992 | Rec: 0.884 | F1: 0.935 | FPR: 0.007 | FNR: 0.116

Valid - Loss: 0.3385 | Acc: 0.864 | Prec: 0.900 | Rec: 0.818 | F1: 0.857 | FPR: 0.091 | FNR: 0.182

Best Valid F1: 0.857 | Patience: 13/15

Early stopping à l'epoch 32

ÉVALUATION FINALE SUR TEST SET

Loss: 0.6918

Accuracy: 0.700

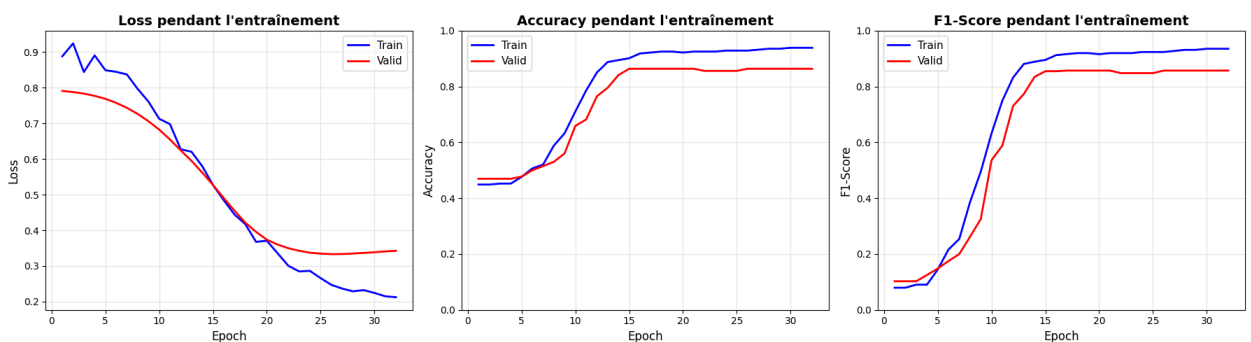
Precision: 0.664

Recall: 0.810

F1-Score: 0.730

FPR: 0.410

FNR: 0.190



```
In [ ]: def show_metrics_per_epoch(history):  
    epochs = range(1, len(history['train_f1']) + 1)  
  
    fig, ax = plt.subplots(figsize=(10, 6))  
  
    ax.plot(epochs, history['train_f1'], 'b-', label='Train F1', linewidth=2)  
    ax.plot(epochs, history['valid_f1'], 'r-', label='Valid F1', linewidth=2)  
    ax.plot(epochs, history['train_acc'], 'b--', label='Train Accuracy', line
```

```

ax.plot(epochs, history['valid_acc'], 'r--', label='Valid Accuracy', linewidth=2)

ax.set_xlabel('Epoch', fontsize=12)
ax.set_ylabel('Score', fontsize=12)
ax.set_title('Métriques par époque', fontsize=14, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
ax.set_ylim([0, 1])

plt.tight_layout()
plt.show()

show_metrics_per_epoch(history)

def show_loss_per_epoch(history):
    epochs = range(1, len(history['train_loss']) + 1)

    fig, ax = plt.subplots(figsize=(10, 6))

    ax.plot(epochs, history['train_loss'], 'b-', label='Train Loss', linewidth=2)
    ax.plot(epochs, history['valid_loss'], 'r-', label='Valid Loss', linewidth=2)

    ax.set_xlabel('Epoch', fontsize=12)
    ax.set_ylabel('Loss', fontsize=12)
    ax.set_title('Loss par époque', fontsize=14, fontweight='bold')
    ax.legend(fontsize=10)
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

show_loss_per_epoch(history)

def show_gradient_per_epoch(history):
    epochs = range(1, len(history['grad_norm']) + 1)

    fig, ax = plt.subplots(figsize=(10, 6))

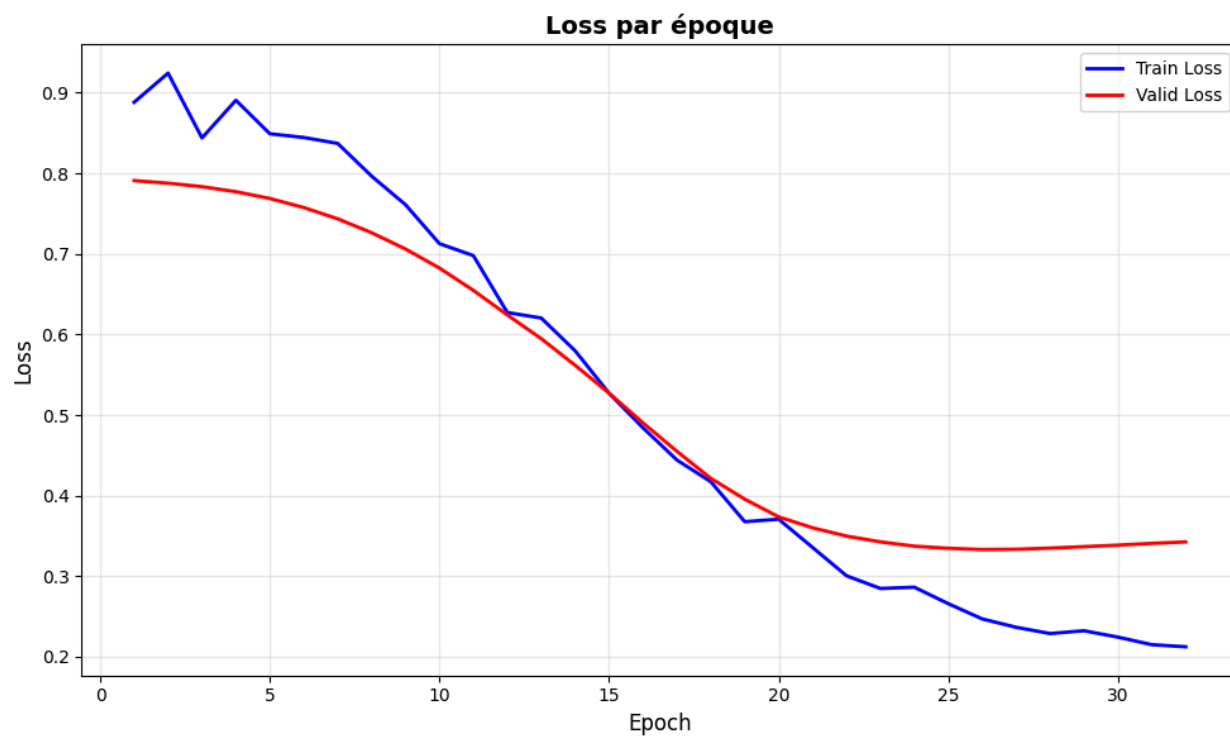
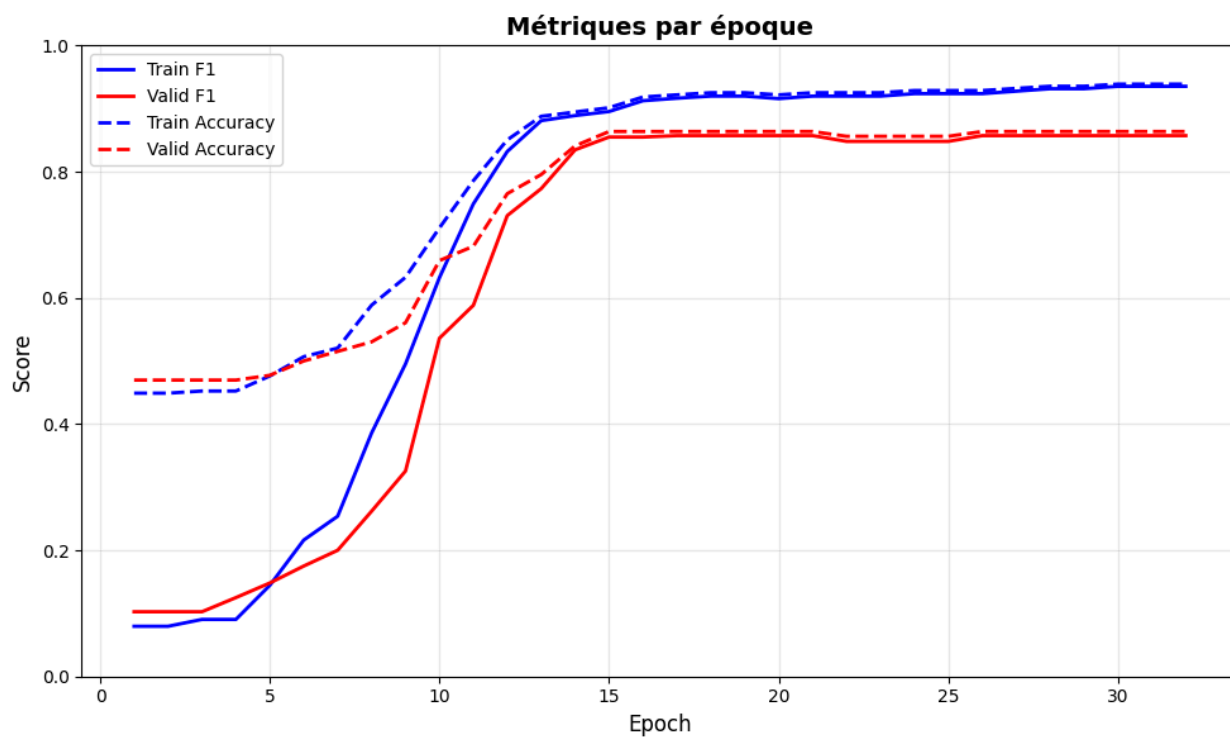
    ax.plot(epochs, history['grad_norm'], 'g-', linewidth=2)

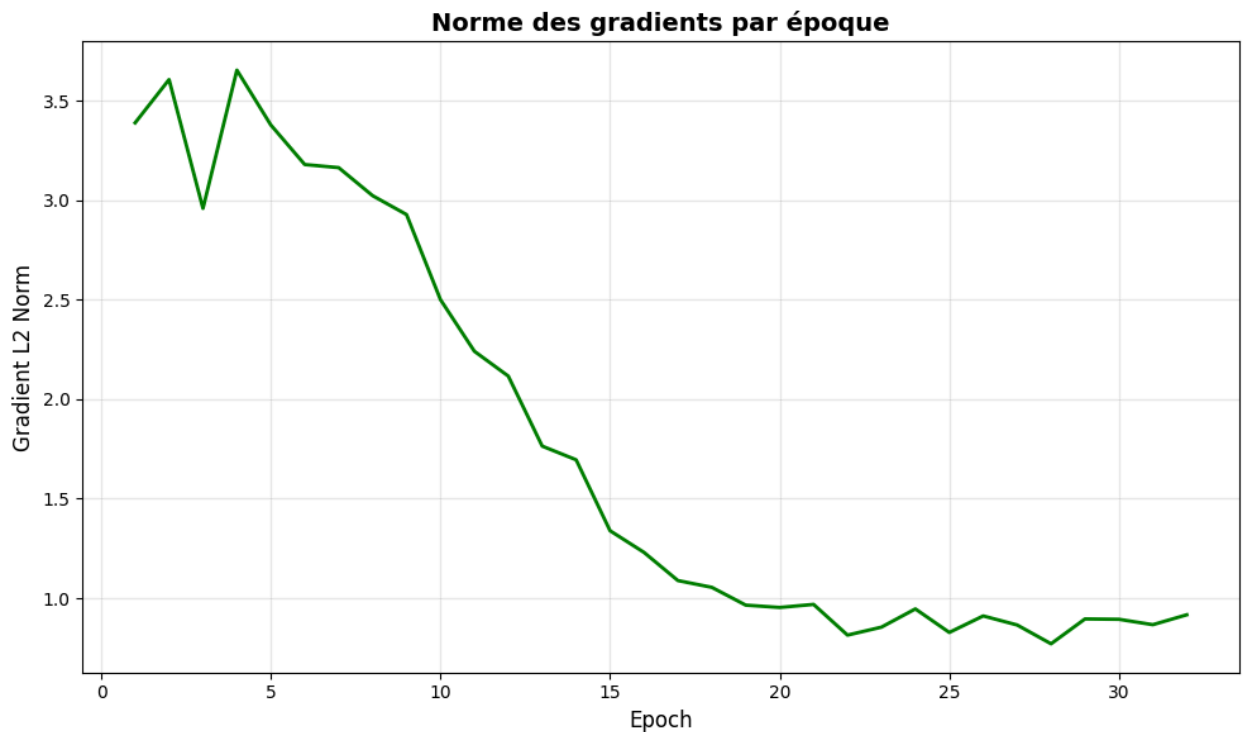
    ax.set_xlabel('Epoch', fontsize=12)
    ax.set_ylabel('Gradient L2 Norm', fontsize=12)
    ax.set_title('Norme des gradients par époque', fontsize=14, fontweight='bold')
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

show_gradient_per_epoch(history)

```





```
In [10]: class PyTorchTransformerEncoder(nn.Module):
def __init__(self, d_model, num_heads, num_layers, dim_feedforward, dropout):
    super().__init__()

    encoder_layer = nn.TransformerEncoderLayer(
        d_model=d_model,
        nhead=num_heads,
        dim_feedforward=dim_feedforward,
        dropout=dropout,
        batch_first=True
    )

    self.transformer_encoder = nn.TransformerEncoder(
        encoder_layer,
        num_layers=num_layers
    )

    self.classifier = nn.Linear(d_model, 1)

def forward(self, x):
    # x: (batch, seq_len, d_model)
    x = self.transformer_encoder(x)
    x = x.mean(dim=1) # Pooling global sur la séquence
    x = self.classifier(x)
    return x.squeeze(-1)

N_FFT = 1024
HOP_LENGTH = 64
WINDOW_DURATION = 2.0
```

```

SR = 22050 # Assumé

# Pour d_model=64, num_heads=8
FMIN = 100
FMAX = 164 # 64 bins = 64 * (SR / N_FFT) Hz*

class PyTorchTransformerEncoder(nn.Module):
    def __init__(self, input_dim, d_model, num_heads, num_layers, dim_feedforward):
        super().__init__()

        assert d_model % num_heads == 0, f"d_model={d_model} must be divisible by num_heads"

        self.input_projection = nn.Linear(input_dim, d_model)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=num_heads,
            dim_feedforward=dim_feedforward,
            dropout=dropout,
            batch_first=True
        )

        self.transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers)
        self.classifier = nn.Linear(d_model, 1)

    def forward(self, x):
        # x: (batch, freq, time)
        x = x.transpose(1, 2) # → (batch, time, freq)
        x = self.input_projection(x) # → (batch, seq_len, d_model)
        x = self.transformer_encoder(x)
        x = x.mean(dim=1)
        x = self.classifier(x)
        return x.squeeze(-1)

def get_best_threshold(y_true, y_probs, thresholds=np.arange(0.0, 1.01, 0.01)):
    best_threshold = 0.5
    best_f1 = 0.0

    for threshold in thresholds:
        y_pred = (y_probs >= threshold).int()
        metrics = compute_metrics(y_true, y_pred)

        if metrics['f1'] > best_f1:
            best_f1 = metrics['f1']
            best_threshold = threshold

    return best_threshold, best_f1

@torch.no_grad()
def find_optimal_threshold(model, loader, device):
    model.eval()
    all_probs = []
    all_labels = []

```

```

    for X_batch, Y_batch in loader:
        X_batch = X_batch.to(device)
        outputs = model(X_batch)
        probs = torch.sigmoid(outputs)
        all_probs.append(probs.cpu())
        all_labels.append(Y_batch)

    all_probs = torch.cat(all_probs)
    all_labels = torch.cat(all_labels)

    best_threshold, best_f1 = get_best_threshold(all_labels, all_probs)
    print(f"Seuil optimal: {best_threshold:.3f} (F1={best_f1:.3f})")
    return best_threshold

# Utiliser sur VALIDSET
best_threshold = find_optimal_threshold(model, valid_loader, device)

def count_parameters(model):
    return sum(p.numel() for p in model.parameters())

# Configuration
d_model = 64
num_heads = 8
num_layers = 2
dim_feedforward = 256

# Ajuster input_dim selon votre spectre
freq_bins = 10 # ou 64 après ajustement FMIN/FMAX
time_frames = 625

transformer_model = PyTorchTransformerEncoder(
    input_dim=freq_bins,
    d_model=d_model,
    num_heads=num_heads,
    num_layers=num_layers,
    dim_feedforward=dim_feedforward,
    dropout=0.1
).to(device)

print(f"MLP parameters: {count_parameters(model):,}")
print(f"Transformer parameters: {count_parameters(transformer_model):,}")

import pandas as pd

comparison = pd.DataFrame({
    'Modèle': ['MLP', 'Transformer'],
    'Paramètres': [count_parameters(model), count_parameters(transformer_model)],
    'Valid F1': [0.0, 0.0], # À remplir
    'Test F1': [0.0, 0.0], # À remplir
    'Temps/epoch (s)': [0.0, 0.0] # À mesurer
})

```

```

print(comparison.to_string(index=False))

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1).float()
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-np.log(10

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        # x: (batch, seq_len, d_model)
        x = x + self.pe[:, :x.size(1), :]
        return self.dropout(x)

class FeedForwardBlock(nn.Module):
    def __init__(self, d_model, dim_feedforward, dropout=0.1):
        super().__init__()
        self.linear1 = nn.Linear(d_model, dim_feedforward)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)
        self.linear2 = nn.Linear(dim_feedforward, d_model)
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x):
        residual = x
        x = self.linear1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.linear2(x)
        x = self.dropout(x)
        x = self.norm(x + residual)
        return x

class MultiHeadAttentionBlock(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        self.attention = nn.MultiheadAttention(
            d_model,
            num_heads,
            dropout=dropout,
            batch_first=True
        )
        self.norm = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

```

```

def forward(self, x):
    residual = x
    attn_output, _ = self.attention(x, x, x)
    x = self.dropout(attn_output)
    x = self.norm(x + residual)
    return x

class EncoderBlock(nn.Module):
    def __init__(self, d_model, num_heads, dim_feedforward, dropout=0.1):
        super().__init__()
        self.attention = MultiHeadAttentionBlock(d_model, num_heads, dropout)
        self.feedforward = FeedForwardBlock(d_model, dim_feedforward, dropout)

    def forward(self, x):
        x = self.attention(x)
        x = self.feedforward(x)
        return x

class EncoderModel(nn.Module):
    def __init__(self, input_dim, d_model, num_heads, num_layers, dim_feedforward, dropout):
        super().__init__()

        assert d_model % num_heads == 0

        self.input_projection = nn.Linear(input_dim, d_model)
        self.pos_encoding = PositionalEncoding(d_model, dropout=dropout)

        self.encoder_blocks = nn.ModuleList([
            EncoderBlock(d_model, num_heads, dim_feedforward, dropout)
            for _ in range(num_layers)
        ])

        self.classifier = nn.Linear(d_model, 1)

    def forward(self, x):
        # x: (batch, seq_len, input_dim)
        x = self.input_projection(x)
        x = self.pos_encoding(x)

        for block in self.encoder_blocks:
            x = block(x)

        x = x.mean(dim=1)
        x = self.classifier(x)
        return x.squeeze(-1)

custom_transformer = EncoderModel(
    input_dim=freq_bins,

```

```

d_model=64,
num_heads=8,
num_layers=2,
dim_feedforward=256,
dropout=0.1
).to(device)

print(f"Custom Transformer parameters: {count_parameters(custom_transformer):,},

```

Seuil optimal: 0.270 (F1=0.894)

MLP parameters: 49,793

Transformer parameters: 100,737

Modèle	Paramètres	Valid F1	Test F1	Temps/epoch (s)
MLP	49793	0.0	0.0	0.0
Transformer	100737	0.0	0.0	0.0

Custom Transformer parameters: 100,737

In []: